UNIVERSIDADE FEDERAL DE RONDONÓPOLIS
INSTITUTO DE CIÊNCIAS EXATAS E NATURAIS
CURSO DE BACHARELADO EM SISTEMAS DE INFORMAÇÃO

CHRISTIAN CESAR RODRIGUES SANTOS

ARQUITETURA DE MICROSSERVIÇOS: PROJETOS, PADRÕES E PRINCÍPIOS

Rondonópolis
2022

CHRISTIAN CESAR RODRIGUES SANTOS

ARQUITETURA DE MICROSSERVIÇOS: PROJETOS, PADRÕES E PRINCÍPIOS

Trabalho de Conclusão de Curso apresentado ao Instituto de Ciências Exatas e Naturais da Universidade Federal de Rondonópolis como requisito parcial para obtenção do título de bacharel em Sistemas de Informação.

Orientador(a): Maikon Adiles Fernandez Bueno

Coorientador(a): Mara Andréa Dota

Rondonópolis
2022

CHRISTIAN CESAR RODRIGUES SANTOS

ARQUITETURA DE MICROSSERVIÇOS: DESIGN, PATTERNS AND PRINCIPLES

Trabalho de Conclusão de Curso apresentado ao Instituto de Ciências Exatas e Naturais da Universidade Federal de Rondonópolis como requisito parcial para obtenção do título de bacharel em Sistemas de Informação.

Aprovado em: 01 de dezembro de 2022.

Banca Examinadora:

Prof.(a) Dr.(a) Maikon Adiles Fernandez Bueno
Orientador(a)
UFR

Prof.(a) Dr.(a) Mara Andréa Dota
Examinador(a)
UFR

Prof.(a) Dr.(a) Cleyton Slaviero
Examinador(a)
UFR

LISTA DE ABREVIATURAS E SIGLAS

SRP Single Responsibility Principle
OCP Open Closed Principle
LSP Liskov Substitution Principle
ISP Interface Segregation Principle
DIP Dependency Inversion Principle
DDD Domain-Driven Design
DTO Data Transfer Object
AWS Amazon Web Service

RESUMO

Com o aumento da demanda de softwares, a cobrança por sistemas com entregas mais rápidas, inteligentes e comunicação entre multi dispositivos ou sistemas distintos é crescente, requerendo cada vez mais o consumo e o compartilhamento de dados, os sistemas evoluem com essa necessidade de novas arquiteturas, metodologias e tecnologias. A construção de um software é difícil por conter tantas possibilidades e tecnologias disponíveis, engenheiros e técnicos investem uma grande parcela do tempo focados na tecnologia e se esquecem do principal, o domínio do negócio. O intuito deste trabalho não é demonstrar todas as arquiteturas ou tecnologias disponíveis, mas sim, entregar compreensão sobre o que é a arquitetura de Microsserviços e como o *Domain Driven Design*, Princípios SOLID, Arquitetura Hexagonal e Padrões de Projeto podem ajudam a manter o foco no domínio e ter um projeto consistente, independente da arquitetura.

Palavras-chave: Microsserviços, Monolíticos, SOLID, Domain Driven Design, Serviços, Repositórios.

ABSTRACT

With the increase in software demand, the demand for systems with faster, smarter deliveries and communication between multiple devices or different systems is growing, requiring more and more consumption and data sharing, systems evolve with this need for new architectures. , methodologies and technologies. Building software is difficult because it contains so many possibilities and technologies available, engineers and technicians invest a large part of their time focused on technology and forget about the main thing, the

business domain. The purpose of this work is not to demonstrate all available architectures or technologies, but rather to deliver an understanding of what the Microservices architecture is and how Domain Driven Design, SOLID Principles and Design Patterns can help keep the focus on the domain and have a consistent design, regardless of architecture.

Keywords: Microservices, Monoliths, SOLID Principles, Domain Driven Design, Services, Repositories.

SUMÁRIO

1 INTRODUÇÃO 7

2 REVISÃO DE LITERATURA 10

2.1 ARQUITETURA DE SOFTWARE 10

2.2 ARQUITETURA MONOLÍTICA 11

2.2.1 VANTAGENS 12

2.2.2 DESVANTAGENS 13

2.3 ARQUITETURA DE MICROSSERVIÇOS 14

2.3.1 VANTAGENS 15

2.3.1 DESVANTAGENS 17

2.4 ARQUITETURA MONOLÍTICA VERSUS MICROSSERVIÇOS 17

2.5 ARQUITETURA HEXAGONAL 18

2.6 DOMAIN DRIVEN DESIGN 19

2.6.1 LINGUAGEM UBÍQUA (UBIQUITOUS LANGUAGE) 20

2.6.2 CONTEXTO DELIMITADO (BOUNDED CONTEXT) 20

2.6.3 MAPA DE CONTEXTO (CONTEXT MAP) 21

2.7 PRINCÍPIOS SOLID 22

3 METODOLOGIA 23

3.1 DESENVOLVIMENTO 24

3.1.1 ANÁLISE 24

3.1.2 APLICANDO O DDD 25

3.1.3 CODIFICAÇÃO 27

3.1.3.4 DESENVOLVENDO COM A ARQUITETURA MONOLÍTICA 28

3.1.3.5 DESENVOLVENDO COM A ARQUITETURA DE MICROSSERVIÇOS 37

4 RESULTADOS 47

5 CONCLUSÃO 53

6 REFERÊNCIAS 54

1 INTRODUÇÃO

Segundo Fowler (2003) em “Patterns of Enterprise Application Arquitecture”, construir softwares é difícil e a medida que crescem se tornam mais ainda. Em um mundo cada vez mais conectado, aplicações que suportam linguagens de programação e tecnologias distintas se tornaram cada vez mais necessárias. A demanda e a entrega se tornaram cada vez mais difíceis de se cumprir. Equipes tendem a tomar cuidado maior para que uma nova funcionalidade não quebre com a outra existente e isso em uma estrutura centralizada onde várias vezes equipes desenvolvem um mesmo projeto é um trabalho árduo e demorado.

Richardson em “Microservice Patterns”, cita o trabalho de Fred Brooks, mencionando que na engenharia de software não existem "balas de prata", apesar de muitos desenvolvedores ainda disporem de tempo para discutir “sobre suas balas de prata". Portanto, os Microsserviços não estão imunes ao efeito "bala de prata".

A definição de uma arquitetura é tão importante quanto a codificação, implementação ou linguagem de programação, pois, definindo a arquitetura de um sistema, diretamente implica nas etapas posteriores, ficando bem definido qual linguagem, metodologia, padrões de projetos a serem aplicados para solucionar tais desafios ao decorrer da execução do projeto.

Segundo Richardson, a essência da arquitetura é a decomposição entre duas partes, os elementos e relacionamentos, e ressalta que essa decomposição é importante, pois facilita a divisão do trabalho onde pessoas e equipes possam contribuir para o projeto e definir como os elementos devem se comunicar. Erder, Pureur e Woods(2015) definem a arquitetura de software como sendo *“estruturas fundamentais de um sistema”*.

Para Summerville (2011), "O projeto arquitetônico é um processo criativo onde se projeta uma organização de sistema que irá satisfazer os requisitos funcionais e não funcionais de um sistema", influenciando nas "Propriedades de um sistema, como desempenho, segurança e disponibilidade."

Para Martin Fowler (2003) a melhor definição de arquitetura vem de Raph Johnson, o qual afirma que:

“[...] a arquitetura é uma coisa subjetiva, uma compreensão compartilhada do design de um sistema pelos desenvolvedores especialistas em um projeto. Comumente, esse

entendimento compartilhado está na forma dos principais componentes do sistema e como eles interagem. Também se trata de decisões, pois são as decisões que os desenvolvedores desejam acertar desde o início porque são percebidas como difíceis de mudar. A subjetividade também entra aqui porque, se você achar que algo é mais fácil de mudar do que você pensava, então não é mais arquitetônico. No final das contas, a arquitetura se resume às coisas importantes – seja o que for.”.

Portanto, “a arquitetura se resume às coisas importantes – seja o que for.”, uma definição precisa e mesclada das demais definições feitas pelo outros autores.

Apesar da arquitetura de um software ser de suma importância, como citado anteriormente, para Evans(2003) em *Domain Driven Design*, a preocupação se inicia antes de definição de qualquer Arquitetura ou tecnologia, pois, independente da infraestrutura ou tecnologia usada, caso a equipe não compreenda a complexidade do próprio domínio, qualquer alteração ou extensão o projeto terá dificuldades em se manter estável e saudável.

Deste modo, o objetivo deste trabalho foi é descrever a Arquitetura de Microsserviços, fazer comparativo com a Arquitetura Monolítica e demonstrar como desenvolver um mesmo tema de sistema, usando a Arquitetura Monolítica e Microsserviços com o auxílio do *Domain Driven Design*, Princípios SOLID, Arquitetura Hexagonal e Padrões de Projeto.

A Arquitetura de Microsserviços se torna um termo bastante discutido entre os desenvolvedores, arquitetos, engenheiros, etc. Como poderá ser visto, há muitos autores contribuindo para um melhor compreendimento e execução desta Arquitetura. Porém, iniciar um projeto ou migrar a Arquitetura de um sistema Monolítico para Microsserviço realmente é necessário ?. Que tamanho o Monolítico precisa para esta migração acontecer ?. Compensa iniciar um projeto em uma Arquitetura de Microsserviços?. A Arquitetura Monolítica está se tornando obsoleta ?.

Estas perguntas têm uma grande influência e justificativa para que este trabalho exista, comparar as duas Arquiteturas, mostrando suas vantagens e desvantagens, e através de um caso de uso, demonstrar ao leitor um material que possa se tomar uma decisão em cima dos estudos já feitos sobre estas Arquiteturas.

Por isso, o trabalho está dividido em três partes, onde a primeira parte traz a Revisão de Literatura, fazendo levantamento bibliográfico dos principais autores sobre o tema de desenvolvimento de software que aborda explicações pertinentes para o projeto. A parte dois fala da Metodologia que rege o trabalho e a terceira parte detalha o Desenvolvimento dos dois projetos, seguido da Análise e Conclusão.

2 REVISÃO DE LITERATURA

2.1 ARQUITETURA DE SOFTWARE

Para compreendermos a arquitetura de software, Erder, Pureur e Woods (2015) definem como “estruturas fundamentais de um sistema”. Summervile (2011) completa que além de estruturar um sistema, influência nas "Propriedades de um sistema, como desempenho, segurança e disponibilidade."

No entanto, Fowler (2003) em uma troca de e-mail com Ralph Johnson em uma tentativa de definir arquitetura, Johnson ao final define que a “Arquitetura é sobre coisas importantes. Seja o que for.”, pois, a equipe pode julgar o que é importante de acordo com suas experiências e partindo disto a equipe manter a arquitetura escolhidas em boas condições.

Segundo Summerville (2011), "O projeto arquitetônico é um processo criativo onde se projeta uma organização de sistema que irá satisfazer os requisitos funcionais e não funcionais de um sistema".

Por mais que o termo “arquitetura de software” dê a entender que o mesmo se trata do projeto de um bloco único de software, armazenado em um único lugar, e auto-contido - *aka* arquitetura monolítica - a ideia por trás da definição transcende tal visão em diferentes aspectos. Uma arquitetura de software pode tratar de diferentes blocos remotos, comunicando-se de diferentes modos, cada qual com um banco de dados diferente, integrando-se ou não com sistemas externos. Tal organização é muito comum com o advento de tecnologias de nuvem juntamente com utilização de microsserviços para explorar a característica de escalabilidade e também diminuir custos na nuvem.

Nesse contexto, no intuito de explorar a utilização da arquitetura de software em diferentes vertentes, nas seções seguintes serão considerados diferentes fatores que influenciam na escolha entre as arquitetura monolíticas ou de Microsserviços.

2.2 ARQUITETURA MONOLÍTICA

Para compreendermos melhor quais os benefícios a escolha de uma arquitetura de Microsserviço acarreta a um sistema e quando aplicá-la, é necessário explicar o que é considerado uma arquitetura monolítica, suas soluções e limitações. Segundo Newman (2019), *"Quando todas as funcionalidades de um sistema tiveram que ser implantadas juntas, consideramos isso um monólito."*, citando que existem 3 tipos de sistemas monolíticos: o sistema de processo único, o monólito distribuído e os sistemas de caixa preta de terceiros.

Independente da atribuição do tipo do sistema monolítico que Newman (2019) citou acima, compreende-se que uma arquitetura monolítica é um sistema centralizado, encarregado da ‘interface’ do usuário, regra de negócio, comunicação com banco de dados e comunicação

com aplicações externas, podendo ser de terceiros.

A Figura 1, que Chris Richardson apresenta em seu livro 'Microservice Pattern' refere-se a um sistema de restaurante monolítico, usa a arquitetura hexagonal, que está explicada no Item 2.5 (página 18) . Ao centro da representação hexagonal encontra-se a regra de negócio, ao seu redor, camadas de interface fazendo o consumo de serviços de terceiros, comunicação com o banco de dados e recebe requisições de dispositivos heterogêneos.

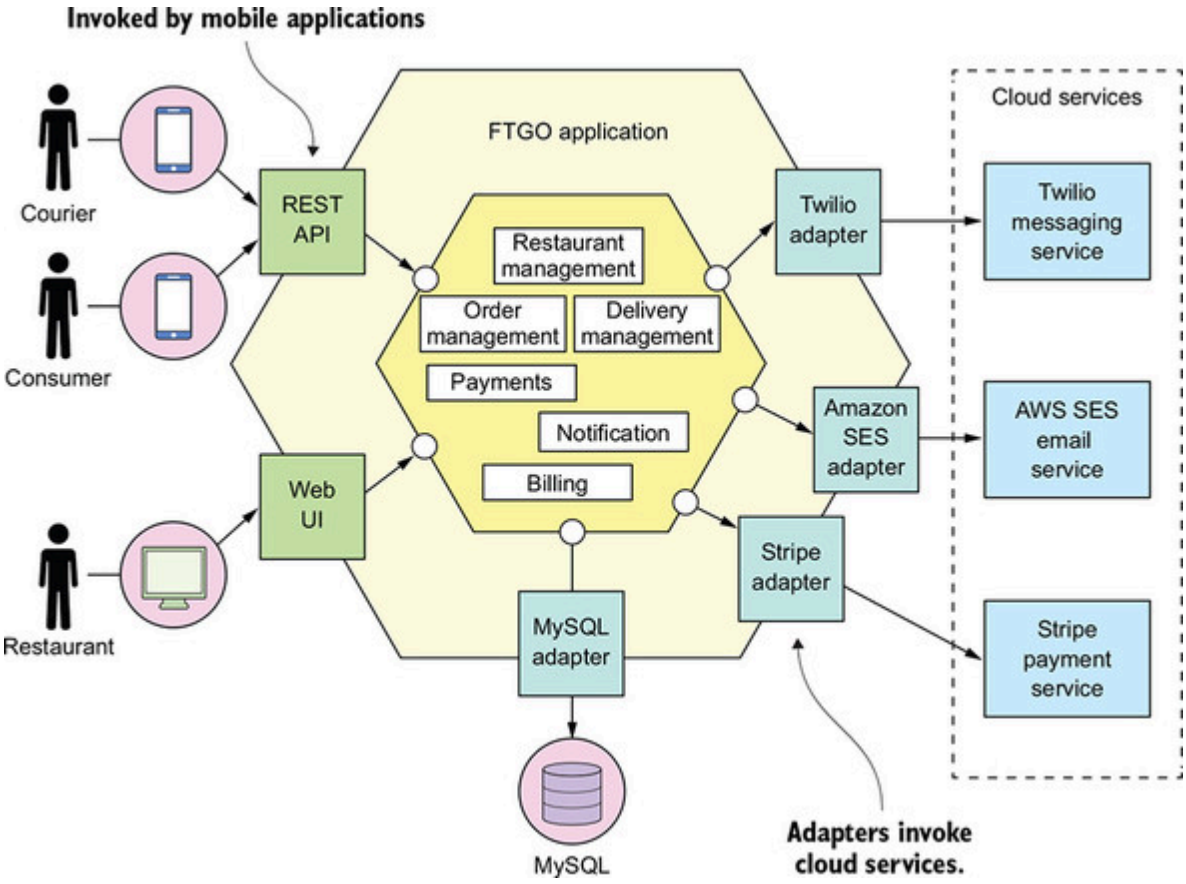


Figura 1. Representação da aplicação Food to Go, Inc. Retirada de Microservices Patterns, Chris Richardson.

A Arquitetura Monolítica é largamente usada, a necessidade de desenvolver um Produto Mínimo Viável (MVP sigla em inglês), com uma equipe pequena e poucos recursos promovem a escolha da Arquitetura, Newman (2019), afirma que, *"Com um monólito, nossas escolhas são muito mais simples, e muitas pessoas gostam dessa simplicidade - todo o código está lá, então use-o!"*.

2.2.1 VANTAGENS

A simplicidade citada por Newman (2019) acima implica na escolha da arquitetura monolítica. Richardson (2018), em seu livro e ‘blog’ pessoal, lista em mais detalhes os benefícios da arquitetura, os quais são: simplicidade no desenvolvimento, facilidade na aplicação de mudanças radicais, simplicidade na execução de testes e na implantação e, por último, facilidade para escalar.

Portanto, o fato de ser simples, como Newman e Richardson citam, se dá ao fato do sistema monolítico se encontrar em apenas um bloco, os Ambiente de Desenvolvimento Integrado (IDE), são projetadas para auxiliar a construção de aplicações completas. Escalar o sistema se torna fácil, pois uma cópia de todo sistema é colocada atrás de um balanceador de carga. Provedores de serviços em nuvem, tais como a Amazon Web Service (AWS), fornece serviços que automatizam esta necessidade de escalabilidade e balanceiam o acesso à aplicação.

2.2.2 DESVANTAGENS

À medida que um monólito cresce, todas as vantagens que adquirira no começo perdem-se gradativamente, ao mesmo tempo, em que se adicionam à balança os novos desafios em manter um grande monólito. Caso o sistema passe do ponto inicial de ser um MVP e se torne um produto final, tendendo a ocorrer com frequência, a engenharia de software ainda não é concreta nas políticas de desenvolvimento e, por isso, há tendências de não ocorrer um posterior replanejamento ou reestruturação do projeto.

Entre as desvantagens citadas por Richardson (2018), a medida que o monólito cresce, a complexidade intimida os desenvolvedores, o desenvolvimento é lento, o caminho do comprometimento à implantação é longo e árduo, o dimensionamento é difícil, a entrega de um monólito confiável é um desafio, preso em uma pilha de tecnologia cada vez mais obsoleta.

Ainda citando Richardson (2018), o desenvolvimento se torna lento e intimidador para desenvolvedores que iniciaram o sistema e ainda mais para os novos, a linha de compreensão e aprendizagem de como o sistema funciona e se comporta torna-se frustrante. O caso piora quando mais desenvolvedores são adicionados ao projeto. A Figura 2 ilustra o caso de 3 (três) grupos de desenvolvimento , cada qual realizando alterações no código, demonstrando o árduo caminho percorrido até a entrega do sistema.

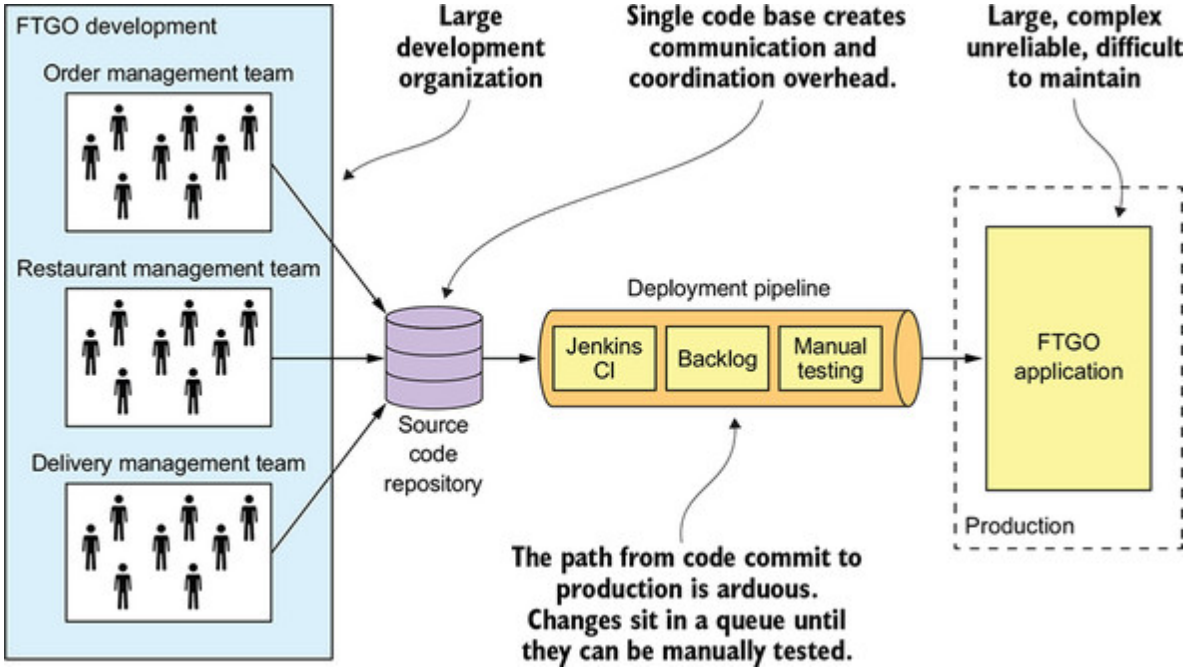


Figura 2. Representando as dificuldades enfrentadas pelo grupo de desenvolvedores ao aplicarem alterações a um monólito.Retirada do livro de Cris Richardson - Microservice Patterns, p.16

2.3 ARQUITETURA DE MICROSSERVIÇOS

O limite de uma arquitetura monolítica, conforme apontado anteriormente, é alcançado quando o gerenciamento se torna custoso e o processo de entrega do sistema se torna problemático para a equipe de desenvolvimento. Tais resultados são caracterizados por Richardson (2018) como "*monolithic hell*" (inferno monolítico), no qual grandes bases de códigos destacam-se negativamente por prejudicar e atrasar a compreensão do sistema pelos desenvolvedores, o que acarreta na demora de novas entregas ou na correção de funcionalidades. Sistemas monolíticos normalmente são versionados em único repositório, o que faz com que a atualização de código seja frequente e que ocorram muitos conflitos. Outro fator importante consiste na escalabilidade, a qual normalmente é custosa, pois escalar um monólito exige a replicação do sistema inteiro. Comumente, a tecnologia utilizada no desenvolvimento sofre pouca atualização, pois o sistema é grande demais para refatorar tudo para a modernização do código.

Diante das desvantagens expostas para manter-se um sistema monolítico, escolher migrar para uma arquitetura de Microsserviços pode ser fundamental para a persistência de uma aplicação no mercado. No entanto, compreender o que é um Microsserviço é a primeira parte deste processo.

Richardson(2018) define Microsserviços como:

É um estilo de arquitetura que decompõe funcionalmente um aplicativo em um conjunto de serviços. Observe que esta definição não diz nada sobre o tamanho. Em vez disso, o que

importa é que cada serviço tenha um conjunto de responsabilidades focado e coeso. Richardson(2018).

Portanto, a arquitetura de Microsserviços, são aplicações completas, com um escopo definido, com única responsabilidade, porém, não são o sistema final. Pois, a aplicação monolítica é dividida em domínios, esses domínios contêm responsabilidades únicas, podendo assim serem desacopladas da aplicação monolítica, convertendo-se a um Microsserviço.

No entanto, é importante salientar outra definição feita por Newman(2019) para a melhor compreensão da arquitetura:

Os Microsserviços são serviços implantáveis de forma independente, modelados em torno de um domínio de negócios. Eles se comunicam entre si por meio de redes e, como opção de arquitetura, oferecem muitas opções para resolver os problemas que você pode enfrentar. Conclui-se que uma arquitetura de Microsserviço é baseada em vários Microsserviços de colaboração. Newman(2019).

Na definição de Newman fica mais óbvio a separação e a responsabilidade de um Microsserviço, e sua contribuição para a aplicação final, como ilustrado na Figura 3, retirada do livro de Richardson (2018).

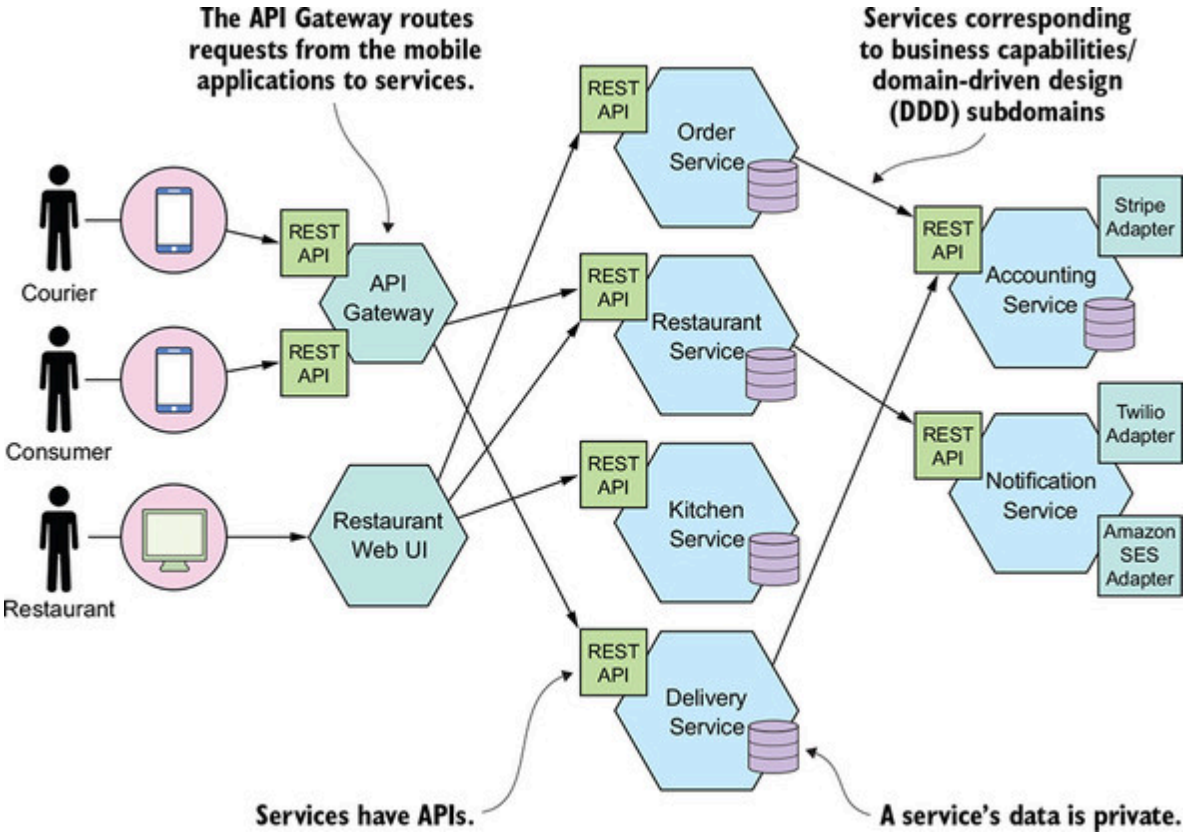


Figura 3. Exemplificando uma arquitetura de microsserviços, com alguns serviços do sistema Food to Go. Retirada do livro de Cris Richardson - Microservice Patterns, p.13

Explicando sobre arquitetura monolítica, Richardson, ilustra o sistema monolítico da empresa Food to Go. Na Figura 3 é possível visualizar a diferença entre as duas arquiteturas e suas organizações, comunicações.
Portanto, sistemas que fazem o uso da Arquitetura de Microsserviços, formam um conjunto de aplicações distribuídas, com responsabilidades definidas, podendo estar localizadas em lugares geograficamente diferentes e cada qual com seu próprio banco de dados, com tecnologias diferentes.

2.3.1 VANTAGENS

A Arquitetura de Microsserviços surge da necessidade de facilitar o desenvolvimento, escalar e gerenciar um sistema. Richardson (2018) lista algumas vantagens importantes:

- Permite a entrega e implantação contínuas de aplicativos grandes e complexos.
- Os serviços são pequenos e de fácil manutenção.
- Os serviços podem ser implantados de forma independente.
- Os serviços são escalonáveis de forma independente.
- A arquitetura de Microsserviço permite que as equipes sejam autônomas.
- Permite fácil experimentação e adoção de novas tecnologias.
- Possui melhor isolamento de falhas.

A lógica por trás de um Microsserviço como podemos perceber, é dividir a responsabilidade entre sistemas menores. Em um sistema menor, a compreensão por parte do desenvolvedor é mais rápida e a entrega de alteração pode ser melhor estipulada. Essa divisão também faz com que alterações em um determinado sistema não afetem diretamente os demais, ou seja, as falhas são isoladas, fator influenciado positivamente na agilidade de identificação e correção das mesmas. A opção de escalar determinados serviços em vez de copiar uma aplicação inteira fazem parte de grupos de vantagens que uma arquitetura de Microsserviço pode ter.

2.3.1 DESVANTAGENS

As desvantagens ligadas aos Microsserviços estão em sua implementação, pois, em um sistema fracamente acoplado, compreender como automatizar integração e entregas contínuas, transações entre serviços se torna um grande desafio. Uma aplicação de microsserviços, tem dificuldades de manter ACID (acrônimo de Atomicidade, Consistência, Isolamento e Durabilidade) de comunicação, pois, compreender, se a demora, dá pelo fato de ser um erro não tratado ou demora na resposta, latência. A lista de vantagens citadas anteriormente, segundo Newman (2019) podem variar, contudo, o autor destaca que nenhuma dessas vantagens são "gratuitas", há um preço a pagar, seja em conhecimento para

implementá- las e/ou custeá-las, e implementá- las deliberadamente podem trazer uma série de prejuízos.

2.4 ARQUITETURA MONOLÍTICA VERSUS MICROSSERVIÇOS

Há uma certa preocupação por parte de Newman (2019), pois, cada vez mais pessoas definem a arquitetura monolítica problemática e determinam em não usar. No entanto, continua a defender em seu livro que a arquitetura monolítica é totalmente válida, pode não ser a melhor escolha para todos os contextos, porém, ressalta que a arquitetura de Microsserviços também não é a solução para o desenvolvimento de todo projeto.

Newman (2019) ressalta a escolha de uma arquitetura há de se analisar pelo contexto, sendo julgado por um grupo de especialista que define o que há de ser considerado importante, assim como Fowler (2003) e Johnson cita “Arquitetura é sobre coisas importantes. Seja o que for.”. Richardson em seu ‘blog’ (microservices.io, https://microservices.io/patterns/monolithic.html), lembra que Netflix, Amanzon e eBay originalmente eram monolíticas. A Netflix em uma palestra, intitulada "Mastering Chaos - A Netflix Guide to Microservices" na QCon San Francisco 2016, demonstra este caso de uso de seu início usando uma arquitetura monolítica e a motivação por trás da transição de arquitetura para Microsserviço.

2.5 ARQUITETURA HEXAGONAL

A arquitetura, como visto anteriormente, ajuda e auxilia o desenvolvedor ou partes interessadas a projetarem um bom software, sendo possível alcançar o objetivo para qual o projeto foi iniciado. Portanto, para Newman (2019), profissionais e o mercado evoluíram criando e adaptando tecnologias, de modo que colaboraram para o sucesso do desenvolvimento de softwares. As arquiteturas, métodos, padrões e princípios apresentados neste projeto fazem parte desta evolução.

A Arquitetura Hexagonal segundo Richardson (2018) é *“estilo de arquitetura que organiza a visão lógica de uma forma que coloca a lógica de negócios no centro.”* onde a *“a aplicação tem um ou mais adaptadores de entrada que tratam de solicitações externas chamando a lógica de negócios.”*. A Figura 4, retirada de seu livro/artigo, ilustra a arquitetura hexagonal.

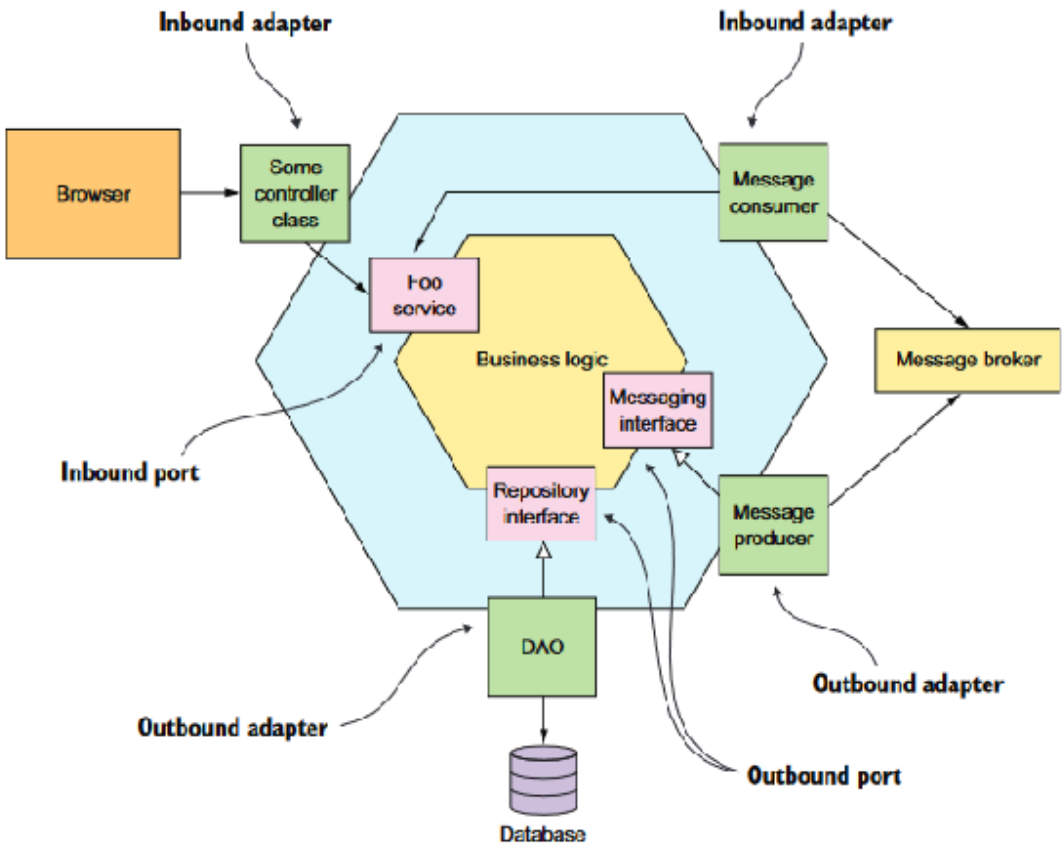


Figura 4. Representação de uma arquitetura hexagonal, retirada do livro de Cris Richardson - Microservice Patterns, p.39.

De forma sucinta, a arquitetura hexagonal faz com que as camadas que estejam em volta da lógica do negócio dependem dela, e não o contrário. Isto é, portas que são interfaces, formam a camada intermediária entre a camada lógica e as demais camadas. Ainda citando Richardson (2018), este é *"um benefício importante"*, pois, *"dissocia a lógica de negócios da lógica de apresentação e de acesso a dados nos adaptadores". A lógica de negócios não depende nem da lógica de apresentação, nem da lógica de acesso a dados."*. Desta maneira, qualquer mudança na tecnologia usada, não terá grandes mudanças em relação à regra de negócio.

Já as arquiteturas Monolíticas e de Microsserviços são classificadas como arquiteturas de visão de implementação por Richardson (2018), porém, ambas com suas diferenças.

- Arquitetura Monolítica: *"é um estilo de arquitetura que estrutura a visão de implementação como um único componente: um único executável ou arquivo WAR."*.
- Arquitetura de Microsserviços: *"é um estilo de arquitetura. Ele estrutura a visão de implementação como um conjunto de vários componentes: executáveis ou arquivos WAR. Os componentes são serviços e os conectores são os protocolos de comunicação que permitem que esses serviços colaborem.*

Em suma, a ***visão de implementação*** compreende uma visão ao todo de aplicação e a ***visão de lógica*** compreende uma visão de organização interna. Dessa forma, a arquitetura hexagonal pode ser incluída dentro de ambas as arquiteturas.

2.6 *DOMAIN DRIVEN DESIGN*

Evans em seu livro *Domain Driven Design*, 2003, cita que a burocracia, objetivos pouco claros e falta de recursos geram problemas ao projeto. Acrescenta haver grande esforço em outros projetos de redes, banco de dados e outras dimensões técnicas de software. Em decorrência, milhares de livros foram escritos resolvendo estes problemas. Vários desenvolvedores cultivaram suas habilidades técnicas e seguiram o avanço tecnológico. Para Evans, a maior complexidade não seria a técnica e sim o próprio domínio. Pois quando a complexidade do domínio não é compreendida e resolvida primeiro, não importa qual infraestrutura ou tecnologia é utilizada, os desenvolvedores, não compreendendo o domínio, não terão capacidade de alterar ou estender o projeto com facilidade e segurança.
A seguir, são apresentadas algumas etapas importantes para desenvolvimento de um projeto orientado pelo DDD definindo a Linguagem Ubíqua, Contexto Limitado e Mapas de Contexto.

2.6.1 LINGUAGEM UBÍQUA (*UBIQUITOUS LANGUAGE*)

Segundo Evans (2003) os especialistas do domínio não compreendem a linguagem ou jargões dos desenvolvedores, mas, por outro lado, são especialistas em sua área. Por outro lado, os desenvolvedores podem compreender e criar abstrações a partir de sua compreensão sobre o projeto. No entanto, podem não ser compreendidas pelos especialistas. Evans acrescenta que continuar um projeto sem uma linguagem comum entre os especialistas e desenvolvedores, pode levar o projeto a uma refatoração destrutiva do código. Evans define a Linguagem Ubíqua como:
Uma linguagem estruturada em torno do modelo de domínio e utilizada por todos os membros da equipe para conectar todas as atividades da equipe ao software. Evans (2003).
Linguagem Ubíqua como mencionado acima, é a linguagem que os especialistas do negócio e desenvolvedores, usam para se comunicar no domínio, definindo seus limites e comportamentos. Obter uma linguagem Ubíqua é importante, pois, além de compreender o domínio, segundo Evans (2003), *"fraquezas do domínio são expostas"*, fazendo o time preparar e desenvolver novas alternativas.

2.6.2 CONTEXTO DELIMITADO (*BOUNDED CONTEXT*)

Após definir a Linguagem Ubíqua, a próxima etapa é definirmos nossos Contextos Delimitados, Evans (2003) define Contextos Delimitados como:
A aplicabilidade delimitada de um determinado modelo. CONTEXTOS DELIMITADOS,

dá aos membros da equipe uma compreensão clara e compartilhada do que deve ser consistente e do que pode se desenvolver de forma independente. Evans (2003)
Como citado anteriormente, a Linguagem Ubíqua ajuda especialistas e desenvolvedores a *"falarem a mesma língua"*, o que leva a diminuir a possibilidade de ambiguidades na regra de negócio, bem como, identificar falhas e dividir os contextos, revelando as responsabilidades dentro do processo da empresa e do futuro sistema. A partir da definição de uma linguagem que todos entendam emerge a possibilidade de também definir e separar os contextos para poderem agir de forma independente, isto é, cada contexto se preocupa exatamente com a sua parte.

2.6.3 MAPA DE CONTEXTO (CONTEXT MAP)

Mapa de contexto, como o nome sugere, mapeia os contextos (domínios) existentes dentro de um projeto. Esta etapa serve para a equipe ter uma visão clara do projeto, observando e analisando as comunicações que esses domínios terão entre si.
Evans (2003) define mapas de contextos como: *"Uma representação dos CONTEXTOS LIMITADOS envolvidos em um projeto e as relações reais entre eles e seus modelos."*. No *Context Map* algumas outras ferramentas são usadas para demonstrar o tipo de contexto e relacionamento. Segundo Evans (2003), algumas delas são:

- *Core Domain* ou Domínio Principal: É *"onde o maior valor deve ser adicionado em seu sistema."*, Evans usa o termo *"Ferva o modelo"*, de modo que encontre o Core do negócio. Terminando sua ênfase dizendo, *"Traga os conceitos mais valiosos e especializados em nítido relevo. Faça o CORE pequeno."*.
- Generic Subdomains ou Subdominios Genéricos: O *Core Domain* é refere se o que há de mais importante na aplicação, aquilo que faz ela existir. Já os *Subdomains* são importantes, porém não constituem o foco, como Evans sugere, *"O modelo está repleto de princípios gerais que todos conhecem ou detalhes que pertencem a especialidades que não são seu foco principal, mas desempenham um papel coadjuvante."*. Estes Domínios e Subdomínios tendem a comunicar-se entre si ou com sistemas externos.
- *Shared Kerne*l: Contextos são limitados a um domínio, focados em solucionar com determinada parte da regra de negócio. No entanto, Evans cita que algumas vezes, equipes podem acabar investindo mais tempo em traduzir e adaptar camadas. Portanto, sugere que a equipe se reúna e compartilhe um núcleo onde ambas as equipes possam usar. Segundo Evans, *"O objetivo é reduzir a duplicação (mas não eliminá-la, como seria o caso se houvesse apenas um CONTEXTO LIMITADO) e tornar a integração entre os dois subsistemas relativamente fácil."*. O autor ainda ressalta que geralmente o Kernel compartilhado pode ser o Domínio Principal ou um conjunto de Subdomínios.

- *Downstream/Upstream*: A comunicação entre dois Contextos Limitados podem ter um relacionamento de Cliente/Fornecedor ou de Conformismo. Um relacionamento Cliente/Fornecedor é caracterizado pelo bom relacionamento entre ambas as equipes dos dois Contextos e procuraram alcançar uma solução juntos. Segundo Evans, esses tipos de relacionamentos costumam dar certo quando fazem parte da mesma gestão. Nesse sentido, o Downstream geralmente se adapta ao Upstream, onde o Upstream não tem nenhuma preocupação sobre como as alterações vão impactar outro Contexto, portanto, o Downstream se conforma e modifica a alteração do Upstream, este tipo de relacionamento caracteriza-se como Conformismo.
- *Anti-corruption*: É uma camada que faz a tradução entre contextos do mesmo sistema ou de sistemas externos. Portanto, uma *"CAMADA ANTICORRUPÇÃO é um meio de vincular dois contextos limitados."*

2.7 PRINCÍPIOS SOLID

`Os princípios de SOLID constituem uma memônica para cinco princípios de desenvolvimento de software, os quais, quando aplicados, agrega flexibilidade, compreensão e manutibilidade ao software. A seguir, o Quadro 1 define cada princípio.`

SRP	***The Single Responsibility Principle***	**Um módulo deve ser responsável por um, e apenas um, usuário ou parte interessada.**
OCP	*The Open Closed Principle*	Você deve ser capaz de estender um comportamento de classes, sem modificá-lo.
LSP	*The Liskov Substitution Principle*	As classes derivadas devem ser substituíveis pelas suas classes bases
ISP	*The Interface Segregation Principle*	Classes não devem ser forçadas a depender de métodos que não usam
DIP	*The Dependency Inversion Principle*	Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender da abstração

Quadro 1. Principios SOLID retirados do livro, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.

Martin (2017), cita que *"Os princípios SOLID nos dizem como organizar nossas funções e estruturas de dados em classes, e como essas classes devem ser interconectadas"*. São princípios aplicados em um nível médio que, *"ajudam a definir os tipos de estruturas de software usadas dentro de módulos e componentes"*, criando estruturas com tolerância a mudanças, de fácil compreensão e que podem ser reutilizadas em outros sistemas.

3 METODOLOGIA

Neste trabalho foi utilizado a pesquisa básica, segundo Silveira (2009, p.34) tem como objetivo "gerar conhecimentos novos, úteis para o avanço da Ciência, sem aplicação prática prevista". Envolve verdades e interesses universais.”. A partir desta natureza de pesquisa, os tipos de pesquisas abordados foram: pesquisa exploratória, bibliográfica e experimental, levando como base Gil (2002) para definir cada uma dessas pesquisas:

Pesquisa exploratória: “Estas pesquisas pretendem proporcionar maior familiaridade com o problema, para torná-lo mais explícito ou a construir hipóteses”.

Pesquisa bibliográfica: “A pesquisa bibliográfica é desenvolvida com base em material já elaborado, constituído de livros e artigos científicos.”

Pesquisa experimental: “De modo geral, o experimento representa o melhor exemplo de pesquisa científica. Essencialmente, a pesquisa experimental consiste em determinar um objeto de estudo, selecionar as variáveis que conseguiriam influenciá-lo, definir as formas de controle e de observação dos efeitos que a variável produz no objeto”.

Este trabalho tem como objetivo explorar as arquiteturas de Microsserviços e Monolíticas, e estudar como o DDD, a Arquitetura Hexagonal, juntamente aos princípios e Padrões de Projetos, podem ajudar o desenvolvimento de software.

Baseado nas definições citadas anteriormente sobre a pesquisa, neste trabalho a pesquisa bibliográfica foi fundamental, nos principais autores referente ao assunto de Microsserviços, DDD, Arquitetura Hexagonal, Princípios e Padrões como, Eric Evans, Chris Richardson, Martin Fowler, Sam Newman, Robert Cecil Martin, Ian Summerville entre outros.

Com o objetivo de utilizar as arquiteturas e princípios descritos nos capítulos anteriores, foi desenvolvida uma aplicação, como prova de conceito, para a aquisição de pacotes de viagens. A escolha de tal tema se deu devido à quantidade de integrações com serviços externos que esse tipo de aplicação demanda, bem como, pela clara definição de separação de domínios que a mesma possui, fato que facilita a aplicação dos conceitos e sua demonstração didática.

3.1 DESENVOLVIMENTO

3.1.1 ANÁLISE

A primeira etapa do projeto, foi compreender como funcionava a compra de um pacote de viagem. Com isso, foi necessário pesquisar como os principais softwares de venda de passagens aéreas, hospedagens de hotéis e pacotes, disponibilizam a pesquisa, listam e finalizam a venda.

A segunda etapa foi procurar uma API *(Application Programming Interface)* que fornecesse dados reais o mais próximo possível. Amadeus é um conjunto de serviços, que disponibiliza dados reais de voos, hotéis, informações de destinos, recomendações, inteligência artificial,

entre outras utilidades. Além das APIs, o Amadeus também disponibiliza um SDK *(Software Development Kit)* para uma grande variedade de linguagens, facilitando a implementação. Amadeus pode ser acessada através do site https://developers.amadeus.com/ e https://github.com/amadeus4dev/amadeus-node
A terceira etapa foi compreender a API, seus métodos e retornos. O serviço disponibiliza todos seus recursos, com isto, foi necessário mapear somente os recursos necessários para o projeto.
Com este mapeamento de recursos finalizados, o próximo capítulo abordará como foi aplicado o *"Domain Driven Design"* no projeto.

3.1.2 APLICANDO O DDD

Como abordado em Linguagem Ubíqua**,** em um projeto instaurar uma Linguagem Ubíqua é a primeira etapa para aplicarmos o DDD, pois, com uma linguagem única entre a equipe e sem ambiguidade de definições, há menores chances de má compreensão sobre o que se trata o projeto e também diminui o risco de erros.
Com isso em mente, a Quadro 02, demonstra como o projeto foi dividido e suas respectivas descrições.

Linguagem Ubíqua	
Domínio	Descrição
Usuário	Responsável pela gestão do usuário, como cadastro, atualização do cadastro, autenticações, compras e consultas de pacotes pagos.
Hotel	Domínio responsável pelas consultas de hotéis disponíveis, persistência de todos os dados necessários para confirmar as diárias.
Voo	Domínio responsável pelas consultas de voos disponíveis, persistência de todos os dados necessários para confirmar a compra do voo.
Pacote	Domínio responsável pela montagem de pacotes aos usuários, este domínio usa os dois domínios Voo e Hotel, para montar e persistir os dados necessários para efetivar a transação.
Autenticação	Domínio responsável pela autenticação do usuário.
Pagamentos	Domínio responsável pelo pagamento das passagens, hospedagens e pacotes.

Quadro 02. Representação da divisão dos domínios e suas responsabilidades.

A próxima etapa é definir nossos Contextos Delimitados, definindo os limites de nossos contextos, prevenimos que um domínio acaba absorvendo responsabilidades que não fazem parte. Nesse sentido, são discriminados no Quadro 3 os contextos e suas responsabilidades.

Contexto Delimitado	
Domínio	Descrição
Usuário	Realiza o registro do cadastro. Realiza pesquisas de passagens, hotéis e pacotes através dos códigos do International Air Transport Association (IATA) . Visualize suas ordens de compras.
Hotel	Realiza buscas de hotéis através dos códigos do International Air Transport Association (IATA). Faz reserva do hotel. Faz pagamento da reserva.
Voo	Realiza buscas de passagens áreas através dos códigos do International Air Transport Association (IATA). Realiza o pagamento da passagem.
Pacote	Realiza buscas de pacotes através dos códigos do International Air Transport Association (IATA) . Faz pagamento do pacote de viagem.
Autenticação	Realiza a autenticação do usuário, provido por serviço próprio. Concede acesso aos usuários devidamente validados.
Pagamento	Realiza pagamentos de passagens, hospedagens e pacotes. Realiza validação dos dados dos cartões. Retorna à informação do pagamento realizado.

Quadro 03. Representação dos Contextos Delimitados.

E por fim, a Figura 5, representa o Mapa de Contexto da nossa aplicação mostrando a comunicação e o relacionamento que cada domínio tem com os demais.

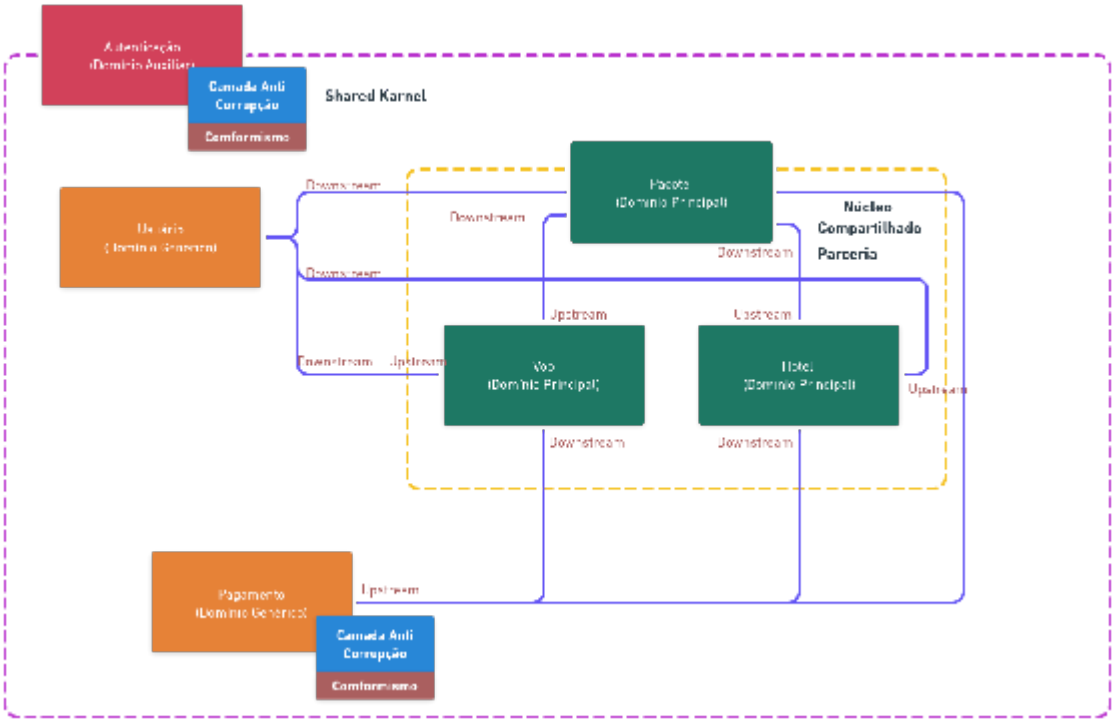


Figura 05. Representação do Mapa de Contexto.

Neste projeto, os domínios de Autenticação e Pagamento não entraram na implementação do projeto em si, pois, há várias estratégias de onde e como podem ser implementadas esses domínios.

3.1.3 CODIFICAÇÃO

Com o intuito de demonstrar as vantagens e desvantagens da utilização da arquitetura de microsserviços, foram desenvolvidos dois projetos, que possuem casos de uso comuns, no entanto, foram implementados utilizando arquiteturas distintas.
Ambos os projetos, 3.1.3.4 Desenvolvendo com a Arquitetura Monolítica e 3.1.3.5 Desenvolvendo com a Arquitetura de Microsserviços, foram desenvolvidos com base em nosso modelo de domínio abordado no tópico anterior, 3.1.2 Aplicando o DDD.
As tecnologias usadas não estão ligadas com a metodologia ou arquitetura, portanto, as tecnologias escolhidas tem a ver com uma experiência pessoal do autor e/ou necessária para o desenvolvimento. As tecnologias usadas para ambos os projetos foram:

- Linguagem de programação: JavaScript/TypeScript;
- Banco de dados: MongoDB Atlas;
- Protocolos de comunicação: HTTP 1.1 (API RESTFul) e HTTP 2.0 (gRPC);
- Mapeamento objeto-relacional (ORM): Prisma

3.1.3.4 DESENVOLVENDO COM A ARQUITETURA MONOLÍTICA

Como citado em 2.3 Arquitetura Monolítica, quando a implementação de um sistema se encontra dentro de grande bloco, isso é caracterizado como uma arquitetura monolítica. A Figura 06 representa como está arquiteturalmente representada a aplicação, no núcleo as regras de negócio e, nas extremidades, os demais provedores necessários para o funcionamento da aplicação.
Figura 06. Representação do projeto, usando a Arquitetura Monolítica com a Hexagonal.

Está aplicação é caracterizada como uma API RESTFul, este tipo de aplicação fornece rotas para serem consumidas por Clientes distintos e heterogêneos, istos é, sua interface gráfica pode ser implementada sobre demanda, podendo ser uma aplicação para web, aplicativos para celulares, relógios e internet das coisas (IOT). Faz uso do protocolo de comunicação HTTP 1.1, podendo fazer o consumo das seguintes rotas disponíveis:

Mapeamento de rotas		
Flights	Responsável por pesquisar ofertas de voos, e persistir os dados do voo selecionado pelo	

Mapeamento de rotas		
	usuário.	
Tipo	Rota	Descrição
POST	/flights	Registra voo selecionado pelo cliente.
GET	/flights/:flightId	Retorna informações conforme o identificador informado do voo
GET	/flights	Retorna todos os voos já registrados na aplicação.
GET	/flights/offers	Com base nos dados informados, retorna uma lista de voos disponíveis.
GET	/flights/user/:userId	Retorna todos os voos já registrados de cliente.
Hotels	Responsável por pesquisar ofertas de hotéis, e persistir os dados do hotel selecionado pelo usuário.	
Tipo	Rota	Descrição
POST	/hotels	Registra o hotel selecionado pelo cliente.
GET	/hotels/:flightId	Retorna informações conforme o identificador do hotel.
GET	/hotels	Retorna a todos os hotéis já registrados na aplicação.
GET	/hotels/offers	Com base nos dados informados, retorna uma lista de hotéis disponíveis.
Packages	Responsável por pesquisar ofertas de pacotes de viagens, e persistir os dados de pacote de viagem selecionado pelo usuário.	
Tipo	Rota	Descrição
POST	/packages	Registra o pacote de viagem selecionado pelo cliente.
GET	/packages/offers	Com base nos dados informados, retorna uma lista de pacotes de viagens disponíveis.

Mapeamento de rotas		
Users	Responsável por persistir os dados dos usuários cadastrados na aplicação	
Tipo	Rota	Descrição
POST	/users	Registra dados do usuário
GET	/users/:flightId	Retorna informações de um usuário específico.
GET	/users	Retorna todos os voos já registrados na aplicação.
PATCH	/users	Atualiza o cadastro de um usuário

Quadro 04. Rotas disponíveis para consumo do cliente.

Para a disponibilidade das rotas da Quadro 04, a aplicação é iniciada pelo arquivo ‘*server.ts*’, de modo que a aplicação se inicia ficando ativa, ouvindo as requisições que serão feitas e retornando os dados consultados no banco de dados, como mostra a Figura 07.



Figura 07. Trecho do código que inicia a aplicação.

A função *"routes"* é responsável por encaminhar todas as requisições feitas pelo cliente para os Controladores responsáveis, fazendo assim a comunicação e o retorno adequado com as regras de negócio da nossa aplicação. A Figura 09, demonstra toda a organização da estrutura do código, a organização facilita a pesquisa dos arquivos e compreensão pelos demais desenvolvedores ao incluir novas funcionalidades ou manutenção.

Toda nossa regra de negócio está contida dentro da pasta "modules", onde dentro de cada domínio há a separação de estruturas, separando e demarcando a responsabilidade de cada componente. Cada domínio possui seus "*Controllers*", Objeto de Transferência de Dados (*DTO*, em inglês), *"Entities"*, *"Repositories"*, *"Routes"* e *"Services"*.

Fora da pasta *"Modules"*, encontra-se a pasta *"Shared"*, que mantém componentes compartilhados, disponíveis para que toda a aplicação tenha acesso. Essa organização constitui um modo de centralizar recursos usados por mais de um contexto, fato que fomenta a reutilização de código e evita replicações desnecessárias.

A organização dos arquivos por pasta proporciona uma melhor separação das responsabilidades de cada parte do código. Tal organização busca espelhar a Arquitetura em Camadas, citada em Evans (2003) e também demonstrada na Figura 08. Na Figura 09, a organização do projeto fica mais evidente.

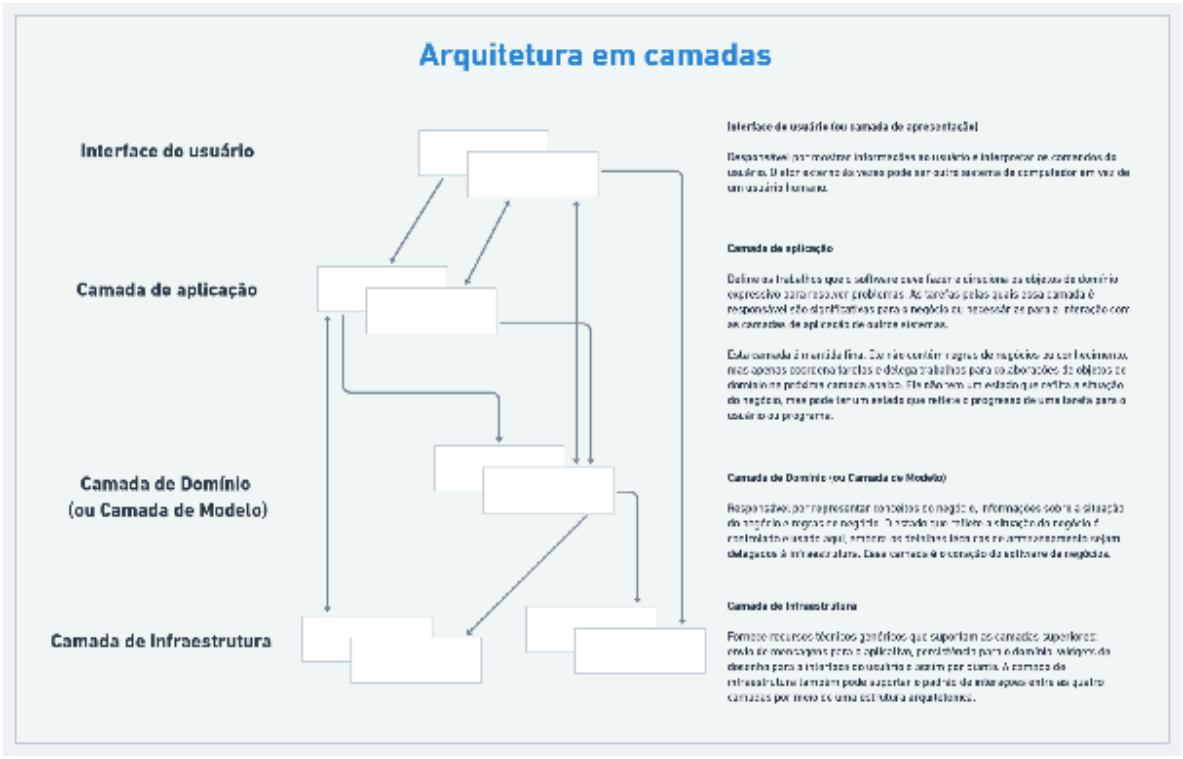


Figura 08. Arquitetura em camadas, ilustração baseada no livro Evans (2003).



Image 09. Estrutura da organização do código.

A aplicação foi desenvolvida fazendo o uso dos Princípios SOLID, para demonstrar a aplicabilidade destes princípios no projeto, usarei o Domínio de Voo como um exemplo.

Na Figura 10, os princípios de Princípio da responsabilidade única e Princípio da inversão da dependência, são aplicados. Na linha 9 podemos ver que há uma injeção de dependência do tipo *"IFlightProvider"*, onde toda e qualquer classe que provê serviço de voo possa ser usada caso implemente o contrato, *"IFlightProvider"*. A classe em si, respeita o primeiro Princípio SRP - classe é responsável por apenas uma função. Nesse caso, a classe *"FlightOfferSearchService"* é responsável apenas pela pesquisa de voos de acordo com os parâmetros passados.

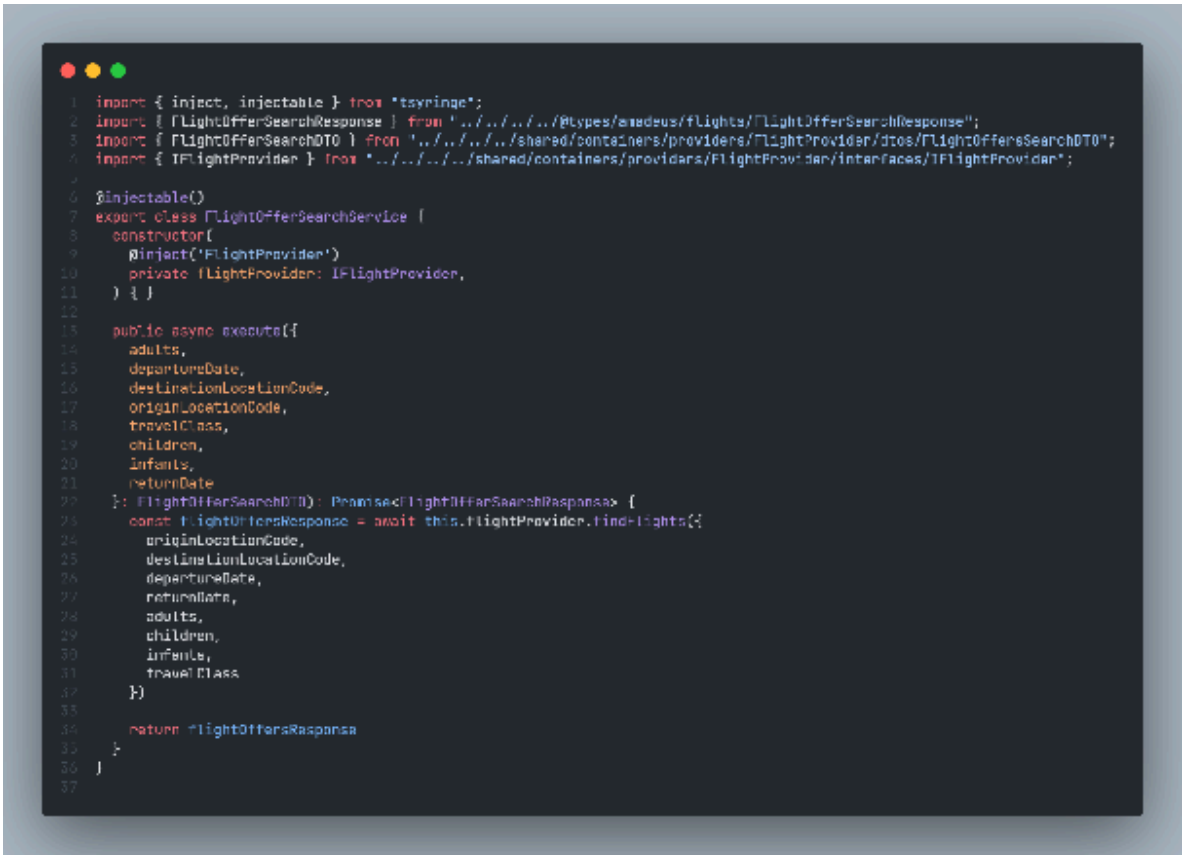


Figura 10. Trecho de código demonstrando a aplicabilidade dos Princípio da responsabilidade única e Princípio da inversão da dependência.

A Figura 10 demonstra a Injeção de Dependência a partir de qualquer classe que implemente, a *"Interface IFlightProvider"*. A *"Interface"* serve como um contrato, de modo que, ao usar a interface a classe deve implementar obrigatoriamente seus métodos, sem modificar a entrada de dados ou a saída. Além de a "Interface" ser um contrato, segundo o Princípio de Segregação de Interface, a "Interface" funciona como algo especializado, de modo que, ao invés de fazer um contrato onde todos os métodos para repositório estão contidos apenas dentro de um arquivo, cria-se "Interfaces" especializadas, para lidar com este contexto. A Figura 11, demonstra o ISP, onde uma "Interface" demonstra um contrato específico dos métodos usados na aplicação para persistir dados, que deve ser implementada sempre quando um novo provedor for ser implementado.



```
import { CreateFlightRequestDTO } from "../../dtos/CreateFlightRequestDTO"
import { Flight } from "../../entities/FlightEntity"

export interface IFlightsRepository {
  create(data: CreateFlightRequestDTO): Promise<Flight>
  findByFlightId(flightId: string): Promise<Flight | null>
  findAll(): Promise<Flight[]>
  deleteFlight(flightId: string): Promise<void>
  findByUserId(userId: string): Promise<Flight[]>
}

```

Figura 11. Trecho de código demonstrando a aplicabilidade dos Princípios de Segregação de Interface.

A classe implementando uma *“Interface”*, faz com que, a classe do tipo *“IFlightsRepository”* possa ser substituída por outra classe que implemente a *“Interface IFlightsRepository”*. Além da substituição, a “Interface” obriga a classe a implementar todos os métodos e impede de retornar valores diferentes daqueles pré-estabelecidos pela *“Interface”*. Este princípio é caracterizado como o Princípio da substituição de Liskov. A Figura 12 representa como foi aplicada na classe *“FlightsRepository”*. Dessa maneira, o projeto usa 4 dos 5 Princípios SOLID.



```
import { prisma } from "../../../../../prisma"
import { CreateFlightRequestDTO } from "../../dtos/CreateFlightRequestDTO"
import { Flight } from "../../entities/FlightEntity"
import { IFlightsRepository } from "../interfaces/IFlightsRepository"

export class FlightsRepository implements IFlightsRepository {
  async create(data: CreateFlightRequestDTO): Promise<Flight> {
    const flight = await prisma.flight.create({ data })
    return flight
  }

  async findByFlightId(flightId: string): Promise<Flight | null> {
    const flight = await prisma.flight.findFirst({ where: { id: flightId } })
    return flight
  }

  async findAll(): Promise<Flight[]> {
    const flight = await prisma.flight.findMany()
    return flight
  }
  async deleteFlight(flightId: string): Promise<void> {
    await prisma.flight.delete({ where: { id: flightId } })
  }
  async findByUserId(userId: string): Promise<Flight[]> {
    const flights = await prisma.flight.findMany({
      where: {
        userId,
        isPackage: false
      }
    })

    return flights
  }
}
```

Figura 12. Trecho de código demonstrando a aplicabilidade dos Princípio da substituição de Liskov.

O desenvolvimento desta aplicação iniciou-se baseada no DDD e, à medida que foram necessários, os Princípios SOLID foram aplicados. Três princípios foram aplicados à medida que cada classe era acrescentada. O Princípio da responsabilidade única, Princípio da substituição de Liskov e Princípio de Segregação de Interface. O resultado da estrutura apresentada na Figura 09, tomou forma aos finais do desenvolvimento, pois, ao atingir o propósito da aplicação, que visava a pesquisa e a compra de passagem de voos, hospedagem em hotéis e pacotes de viagem(voo e hospedagem). O código passou por uma reorganização e refatoração de código, de modo que a aplicação seguisse os conceitos do DDD e Princípios SOLID. Como citado por Newman (2019) em 2.2 ARQUITETURA MONOLÍTICA, a Arquitetura Monolítica permite que as modificações radicais sejam possíveis e permite a construção de um Produto Mínimo Viável (MVP).

3.1.3.5 DESENVOLVENDO COM A ARQUITETURA DE MICROSSERVIÇOS

De acordo com a experiência de Richardson (2018), citada em 2.3 ARQUITETURA DE MICROSSERVIÇOS, a Arquitetura de Microsserviços deve ser uma evolução natural da aplicação Monolítica por diversos motivos, entre os quais podem ser citados:

- o sistema não suporta equipes distintas mexendo em uma mesma base de código;
- a IDE se torna lenta;
- o código se torna menos compreensível com a evolução do sistema;
- a tecnologia utilizada pelo sistema se torna obsoleta; ou
- não suporta tecnologias diferentes das quais tem se utilizado, fato que advém devido às diversas dependências de bibliotecas antigas que o sistema tem adquirido durante seu desenvolvimento.

Ao contrário da evolução citada acima, neste estudo de caso a evolução da Arquitetura se deu pelo motivo de testar as dificuldades da implementação de um mesmo projeto em outra Arquitetura. Desse modo, o foco da abordagem apresentada aqui, não são os itens citados anteriormente, mas sim, a análise da dificuldade da implementação dessa Arquitetura, fazendo o uso dos mesmos Princípios SOLID, DDD e Padrões de Projetos.

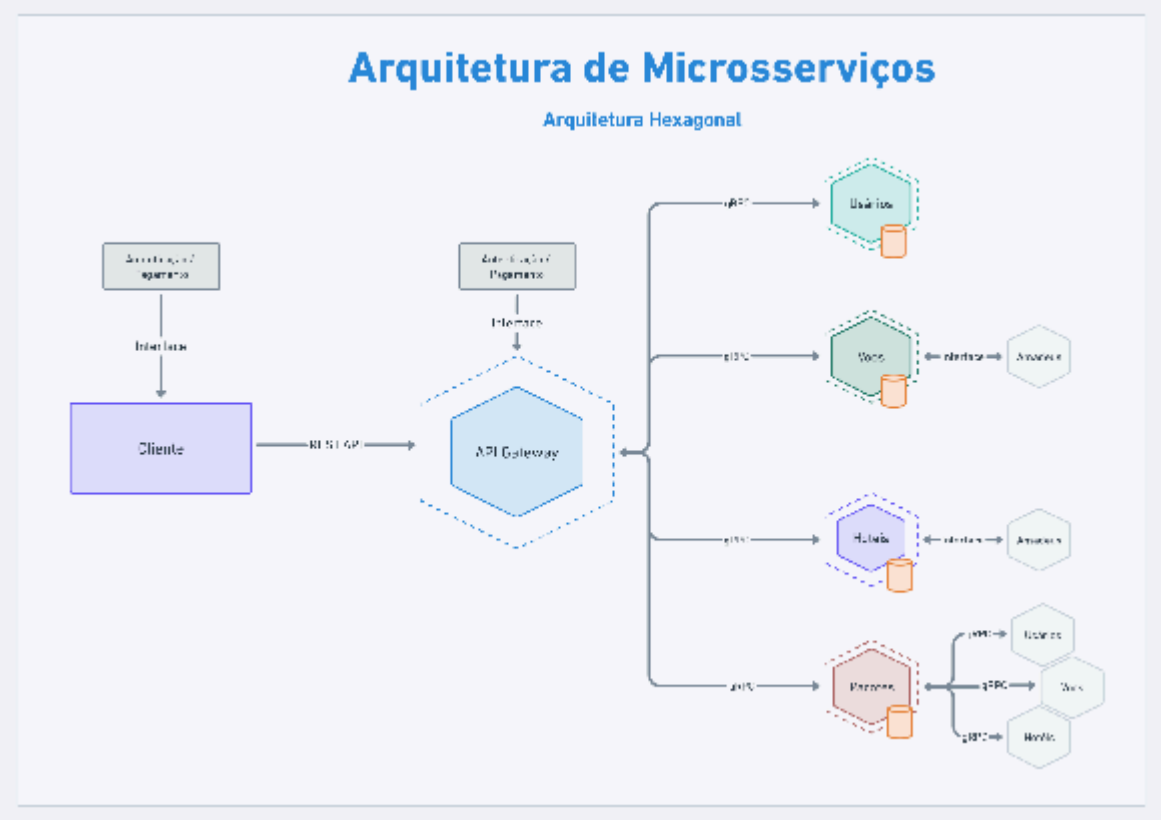


Figura 13. Representação do projeto, usando a Arquitetura de Microsserviços com a Hexagonal.

A Figura 13, que representa a organização do sistema em microsserviços, ilustra que cada Microsserviço possui seu próprio banco de dados, assim cada aplicação não tem interferência direta no banco de dados da outra aplicação, sendo possível apenas o acesso, através das rotas disponibilizadas por cada Microsserviço.

Assim como no 3.1.3.4 DESENVOLVENDO COM A ARQUITETURA MONOLÍTICA, o Cliente faz comunicação com as rotas através do protocolo de comunicação HTTP 1.1 disponível por uma *"API RESTFul"*. Diferente da Arquitetura Monolítica, onde a aplicação se encontra em um único bloco, a Arquitetura de Microsserviços é composta por pequenos sistemas separados e podendo ser gerenciado por equipes diferentes. Dessa maneira, de modo que tenha um ponto que centralize o consumo das demais aplicações por parte do Cliente, sem que o Cliente acesse diretamente cada Microsserviço, o *Padrão "Gateway API"* é usado para manter este ponto de comunicação entre o Cliente e Aplicação, a Figura 13 ilustra a representação da *"API Gateway"*.

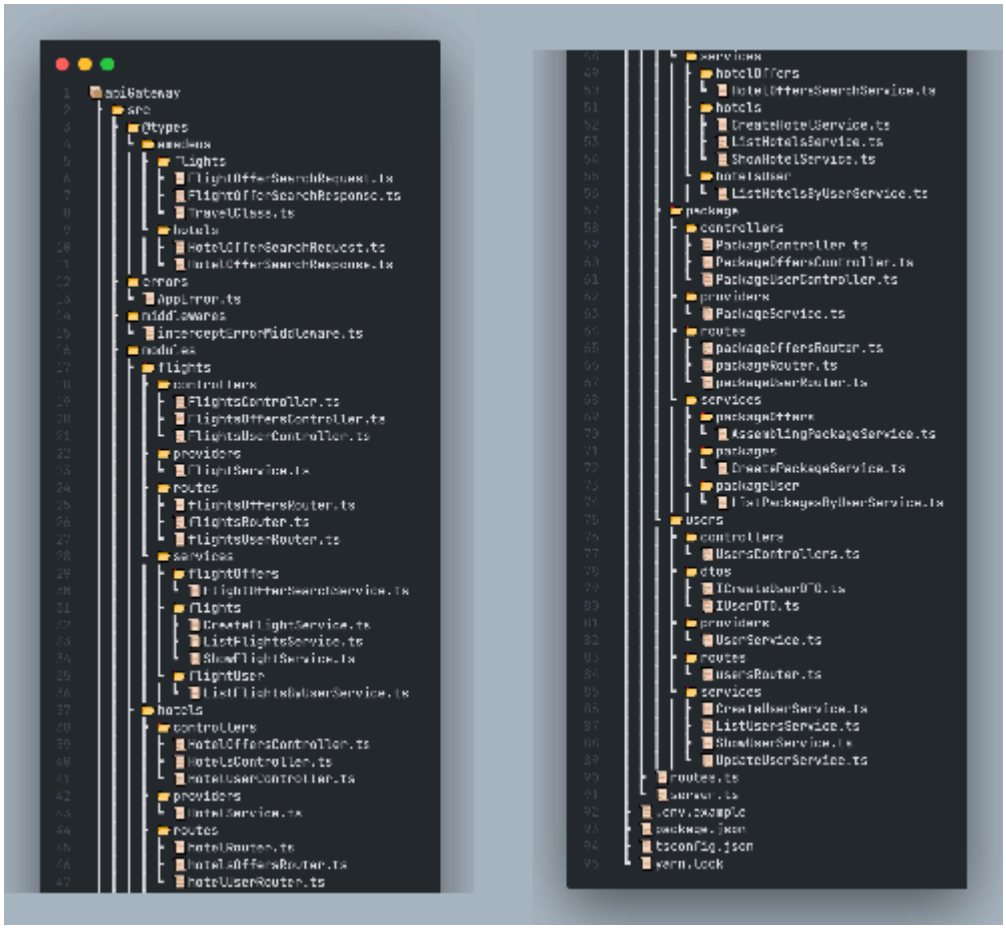


Figura 14. Representação da organização do Padrão "API Gateway" utilizado para desenvolver o microsserviço.

No entanto, a comunicação da *"API Gateway"* com os demais Microsserviços, Figura 15, não ocorre via HTTP 1.1 e sim HTTP 2.0 e *"Google Remote Procedure Call (gRPC)"*. Segundo o site do *"gRPC"* (https://grpc.io/), define-se como:

Uma estrutura de chamada de procedimento remoto (RPC) de código aberto e de alto desempenho moderna que pode ser executada em qualquer ambiente. Ele pode conectar serviços de forma eficiente dentro e entre *data centers* com suporte conectável para balanceamento de carga, rastreamento, verificação de integridade e autenticação.
Dito isso, o “gRPC” é o principal protocolo de comunicação entre a “API Gateway” e os demais Microsserviços, por ser um serviço mais rápido, leve, autenticavel.



Figura 15. Representação dos Quatro Microsserviços.

Cada Microsserviço está disponível no intervalo de portas de 5001 a 5004, assim como demonstrado na Figura 16. A medida que um Microsserviço é iniciado, um servidor, com uma estrutura pré-estabelecida. Na Figura 17, é carregada e disponibilizada em um determinado *IP* e *Porta*. Deste modo, disponibiliza os recursos do Microsserviço a qualquer cliente que tenha o IP:PORTA e o Contrato.



```
import "reflect-metadata";
import "./shared/containers";
import { UsersService } from "dreams-proto-sharing/src/contracts/user/user_grpc_pb";
import { Server, ServerCredentials } from "@grpc/grpc-js";
import { promisify } from "util";
import { UsersServer } from "./modules/protos/implementations/UserServiceProto";
import { prisma } from '../prisma';


const server = new Server()
server.addService(UsersService, new UsersServer())

const bindPromise = promisify(server.bindAsync).bind(server)

bindPromise('0.0.0.0:50052', ServerCredentials.createInsecure())
  .then(async (port) => {
    await prisma.$connect()

    console.log(`listening on ${port}`)
    server.start()
  })
  .catch(async (error) => {
    await prisma.$disconnect();
    console.error(error)
  })
```

Figura 16. Microsserviço Usuário, arquivo responsável por inicializar a aplicação e escutar e responder as requisições dos demais microsserviços.

O Contrato ou Definições estabelecidas em um arquivo *“Proto”*. Figura 17, funciona como um contrato, onde o Servidor e Cliente têm de ter acesso. Esse contrato faz com que o Cliente e Servidor tenham consciência dos métodos disponíveis, parâmetros e tipos de dados que podem ser retornados.



```
syntax = "proto3";

package user;

import "google/protobuf/empty.proto";

message User {
  string id = 1;
  string email = 2;
  string firstName = 3;
  string lastName = 4;
  int64 age = 5;
  string birthday = 6;
}

message UserResponse {
  repeated User user = 1;

}

message UserRequest {
  User user = 1;
}

service Users {
  rpc createUser(UserRequest) returns (UserResponse);
  rpc listUsers(google.protobuf.Empty) returns (UserResponse);
  rpc showUser(UserRequest) returns (UserResponse);
  rpc updateUser(UserRequest) returns (UserResponse);
}
```

Figura 17. Arquivo *Proto File*, definição da estrutura de consumo e resposta do Microsserviços Usuário.

Com o contrato, uma compilação é realizada para a linguagem escolhida, neste caso foram gerados tipagens para a linguagem JavaScript/Typescript, gerando uma *"Interface"*. A partir desta interface, métodos devem ser implementados, pois, estes métodos são responsáveis pela comunicação entre os Microsserviços. Estes métodos correspondem aos métodos declarados em *"service Users"*que informa quais parâmetros são necessários para fazer a requisição e seu possível retorno. A Figura 18 demonstra, esta explicação.



Figura 18. Implementação dos serviços do arquivo Proto.

O Cliente não necessita fazer implementação, como ilustrado na Figura 19, no entanto, precisa ter acesso à compilação feita dos arquivos *"Proto"*. Sem os arquivos de tipificação, o desenvolvimento se torna mais difícil, pois, saber os métodos disponíveis e suas estruturas de dados é essencial para evitar erros nas requisições e respostas.

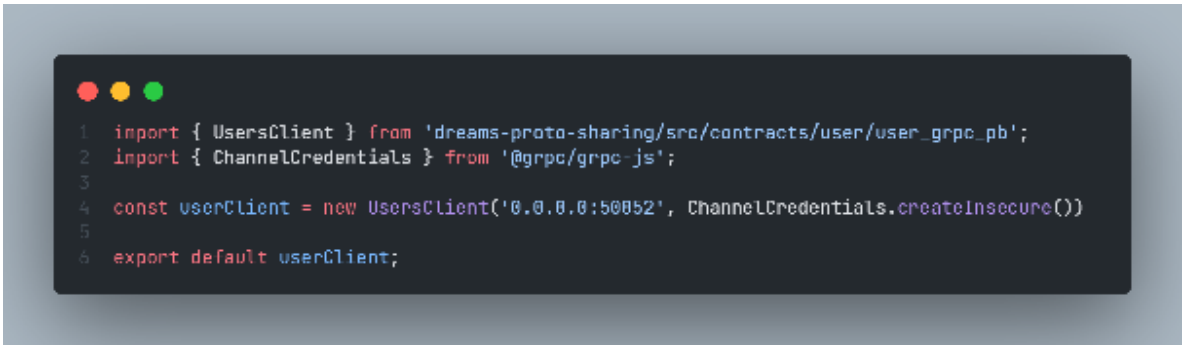


Figura 19. Implementação lado Cliente.

A fim de diminuir a replicação de código e possíveis incoerências, alguns arquivos foram centralizados, como já abordado no 2.6 Domain Driven Design, ter um ponto de compartilhamento não é ruim, no entanto, as mudanças não podem ocorrer de maneira deliberada sem o consentimento da outra equipe ou domínio.

Dessa maneira, um pequeno pacote, uma aplicação independente foi criado, para centralizar os arquivos *"Protos"* e consequentemente os arquivos compilados a partir desses arquivos, facilitando o compartilhamento entre os serviços e diminuindo as replicações de código.

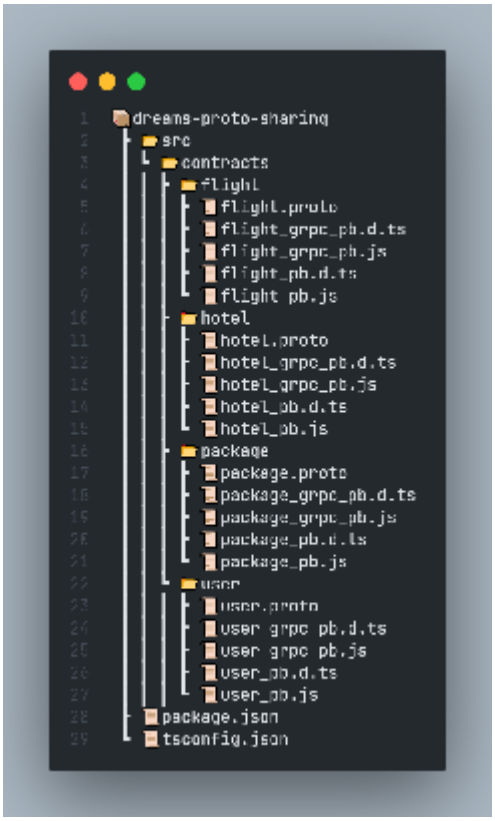


Figura 19.Pacote onde está contido os arquivos “Protos” e arquivos gerados consequente da compilação.

A arquitetura de Microsserviços, como abordado por alguns autores, trata-se de uma evolução da Arquitetura Monolítica. Neste projeto, a mudança de arquitetura não foi um problema, visto que, com o uso do DDD, Princípios SOLID e alguns Padrões de Projeto, foi possível fazer a transição sem que fosse necessário reescrever a regra de negócio.
Cada módulo da aplicação Monolítica tornou-se um Microsserviço e *“API Gateway”*, desta forma, 4 (quatro) Microsserviços (Voos, Hotéis, Pacotes e Usuários) foram criados, o antigo Monolítico foi modificado a ponto de ser apenas um ponto de centralizar o consumo dos demais Microsserviços, o Padrão *“API Gateway”*. A comunicação entre a *“API”* e Microsserviços foi via *gRPC (HTTP 2.0)*, pois, de acordo com o levantamento bibliográfico se tratava de um protocolo rapido, leve e seguro, visto que para a comunicação acontecer, tanto o *“API”* e os Microsserviços devem conter o “contrato” que no caso são os arquivos *“.proto”* e *IP:PORT*, sem estes arquivos ou endereço correto, qualquer acesso externo é dificultoso ou nulo, além do *gRPC* conter camadas de autorização para ser implementadas.
Cada Microsserviços contém seu próprio banco de dados, visando que nenhum serviço teria acesso direto ao Repositório de outro microsserviços diretamente, a Classe *“PackageTipEventHandle”* é usada conforme algum erro ocorra na persistência da criação de um Pacote de Viagem, este método é usado apenas neste Microsserviço, visto que é o único que faz uso dos outros dois Microsserviços Voos e Hotéis. O método “event” da

classe *“PackageTipEventHandle”* é chamado, inativando assim o dado criado anteriormente.



```
1  const flightResponse = await this.flightDreamsProvider.createFlight({
2    itineraries: flight.itineraries,
3    price: flight.price,
4    userId: userId
5  })
6
7  const hotelResponse = await this.hotelProvider.createHotel({
8    hotel: hotel.hotel,
9    offers: hotel.offers,
10   userId
11 }).catch(err => {
12   PackageTripEventHandle.event({
13     eventName: 'CREATE_HOTEL_ERROR',
14     errorDescription: err.message,
15     flightId: flightResponse.id,
16   } as PackageEventParams);
17 })
18
19 const package_ = await this.packageRepository.create({
20   userId,
21   flightId: flightResponse.id,
22   hotelId: hotelResponse!.id,
23   amount,
24   off
25 }).catch(err => {
26   PackageTripEventHandle.event({
27     eventName: 'CREATE_HOTEL_ERROR',
28     errorDescription: err.message,
29     flightId: flightResponse.id,
30     hotelId: hotelResponse!.id
31   } as PackageEventParams);
32 });
```

Figura 20. Trecho do código referente a classe de Criar Pacotes.

Um exemplo seria o Microsserviço de Pacote, ao criar um pacote de viagem a classe “*CreatePackageService*” usa os dois outros Microsserviços, Voos e Hotéis. Na Figura 20, a partir da linha 30 a 34, se faz uma requisição de modo que se sabe um voo para um determinado pacote, e caso ocorra um erro, tanto os dados de voo quanto os da criação do pacote não serão criados. Nas próximas, linhas 36 a 46, caso o Microsserviço de Voo retorne uma resposta positiva, ou seja, um retorno com os dados criados, a próxima etapa é criar a reserva do Hotel. No entanto, caso ocorra um erro no Microsserviço de Hotel ,dispara-se um evento, que de forma assíncrona, envia o identificador (id) do Voo criado anteriormente para mudar seu *“status”* para *“Error”* com o erro retornado, isso permite

controlar e obter melhor informações do motivo de cada *"status"*. O mesmo acontece caso ocorra um erro ao criar um pacote, são disparados dois métodos que enviam para ambos os Microsserviços a atualização do *"status"* e o respectivo erro. Essa comunicação com as demais aplicações acontece através do HTTP 1.0.

4 RESULTADOS

Seguindo as recomendações dos autores, começar pela Arquitetura Monolítica facilitou a transição para a Arquitetura de Microsserviços. Pois, o uso da Arquitetura Monolítica em um pequeno sistema, facilita na compreensão e em grandes mudanças que podem ser feitas, na fase inicial de um sistema. Inicialmente, a ideia do projeto era apenas fazer a construção de um único projeto abordando a Arquitetura de Microsserviços, mas se tornou inviável, pela dificuldade de compreensão do tema de Microsserviços e pela pouca experiência. Desta forma, este primeiro estudo usou uma publicação de Martin Fowler, Monolith First, 2015, em seu blog pessoal descreveu a seguintes observações:

Quase todas as histórias de microsserviços de sucesso começaram com um monólito que ficou muito grande e foi quebrado. Quase todos os casos em que ouvi falar de um sistema que foi construído como um sistema de microsserviço do zero, acabou com sérios problemas.

Porém, com o domínio bem definido, foi observado que uma transição entre Arquiteturas não afetou o que já havia sido desenvolvido, pois, fazendo o uso do *Domain Driven Design*, focando em desenvolver e compreender o domínio da aplicação e na codificação, implementando os Princípios SOLID e Padrões de Projeto a regra de negócio de cada domínio se manteve.

De modo que possa demonstrar as diferenças, dificuldades e soluções encontradas para desenvolver e transferir o projeto entre arquiteturas, será abordada como a ideia de implementar apenas a Arquitetura de Microsserviços, a ideia inicial, deu espaço para que fosse implementado as duas arquiteturas, fazendo assim, a evolução arquitetural do projeto.

Como mencionado acima por Fowler (2015, Monolith First), "Quase todos os casos em que ouvi falar de um sistema construído como um sistema de microsserviço do zero, acabou com sérios problemas.". Com o intuito de abordar apenas a Arquitetura de Microsserviços o primeiro projeto, (https://github.com/christiancesar/dreams-project), com ideias fervorosas de tecnologias a usar, como o uso de *Apache Kafka*, *Docker, Kubernetes, Prometheus*, Integração Contínua e Entrega Contínua com Amazon Web Services, Balanceamento de cargas. O projeto parou em fase inicial da implementação por alguns motivos:

- Dificuldades de compreender *Saga Patterns* abordado por Richardson (2018), o Padrão Saga discute dois métodos: Orquestra e Coreografia, ambos são referente a comunicação entre um ou mais microsserviços usando mensageria para a persistência de dados. O intuito é manter a *ACID (atômicas, consistentes, isoladas e duráveis).*

- Dificuldade em compreender o domínio, sem a compreensão do domínio de compras de passagens e estadias, entender quais dados devem transitar e como cada microserviço se comunicaria ficou confuso.
- Sem conhecimento para criar aplicações com multi contêineres, isto é, cada microsserviço e banco de dados, deveriam estar contidos em contêineres separados. Sem conhecimento para fazer o uso de um gerenciador de contêineres como o *Kubernetes.*

O conhecimento para o desenvolvimento é citado pelos autores como uma desvantagem, pois, é necessário conhecimento e experiência para trabalhar com uma arquitetura que traz flexibilidade, escalabilidade e manutenibilidade como a de microsserviços. É necessário conhecimento mais a fundo do domínio, para compreender seu relacionamento entre as regras de negócio, comunicação, como os dados devem persistir.

Analisando estas informações, percebe-se que o foco foi nas tecnologias que seriam usadas e nas implementações do algoritmo, algo que Evans (2003), afirma que, há grandes esforços para solucionar implementações de projetos de redes e bancos de dados, no entanto, poucos esforços para compreender o domínio do projeto.

Deste modo, este projeto foi divido em duas implementações, onde uma implementa uma arquitetura monolítica com a arquitetura hexagonal e a outra implementa uma arquitetura de microsserviços com a arquitetura hexagonal. Um estudo do domínio mais profundo foi realizado, fazendo o uso do Domain Driven Design, deste modo, com a compreensão do domínio, indiferente seria a arquitetura submetida, o projeto não perderia a ideia, que é proporcionar que um cliente faça a consulta e a compra de pacote de viagem. O desenvolvimento em mais detalhe, consultar o tópico 3.1 DESENVOLVIMENTO.

Portanto, a fim de demonstrar as duas arquiteturas, este projeto ocorreu em um nível de implementação, mostrando o processo de desenvolvimento da regra de negócio, não considerando a escalabilidade ou tecnologias que rodeiam a arquitetura de microsserviços.

Deste modo, como apontada as vantagens e desvantagens de ambas as arquiteturas, ao decorrer da 2. REVISÃO DE LITERATURA. Serão listadas novamente com o intuito de demonstrar se estas vantagens e desvantagens foram sentidas neste projeto.

Arquitetura Monolítica	
Vantagens	**Observação**
Simplicidade ao desenvolver (Todo o código está em um só lugar)	Correto, ao desenvolver o projeto na Arquitetura Monolítica, houve uma facilidade em implementar e testar a comunicação entre os módulos, visto que tudo estava em um único lugar.

Arquitetura Monolítica	
Facilidade na aplicação de mudanças radicais	Correto, ao ter uma compreensão diferente ou aplicar um padrão de projeto ou princípio no código, foi sentida facilidade, pois, poderia promover mudanças de forma rápida em todo o código.
Simplicidade na execução de testes e implantações	Não foi abordado nenhum dos dois conceitos no projeto
Facilidade de escalar	Não foi abordado no projeto
Ambientes de Desenvolvimento Integrado, são projetados para trabalhar com apenas um bloco de código	Correto, em Microsserviços caso queira saber de qual tipo de resposta um método pode retornar, simplesmente saber alguma informação, isso não é possível. De qualquer forma, cada Microsserviço é uma aplicação única, então qualquer referenciamento direto não teria sentido fazer uso da arquitetura.
Desvantagens	**Observação**
As vantagens se tornam desvantagens, a partir que este código se torna grande ou muitas equipes estão mexendo no mesmo repositório.	Não foi possível verificar esta afirmação, pois a codificação não foi extensa. Visto que a máquina de desenvolvimento continha bons recursos de hardware.
Ocorre intimidação de outros desenvolvedores	Não foi possível verificar esta afirmação, pois havia somente um desenvolvedor.
Desenvolvimento lento	Desenvolvimento lento segundo autores do levantamento bibliográfico se dá pelo tamanho do código, falta de estrutura e arquitetura bem definida, falta de compreensão do domínio. O desenvolvimento foi lento por questões de conhecimento do domínio e a falta de compreensão completa dos recursos que as tecnologias detinham.
Microsserviços	
Vantagens	**Observações**
Permite a entrega e implantação contínuas de aplicativos grandes e complexos.	Não foi abordado no projeto.
Os serviços são pequenos e de fácil manutenção.	Correto, com pequenas aplicações em seus contextos delimitados, foi mais focado nas alterações ou inclusões que deveriam ser incluídas no código.
Os serviços podem ser implantados de forma independente.	Correto, cada microsserviço foi implementado de forma independente, tendo seu funcionamento normal. Claro que no caso do microsserviço de Pacotes, que há uma dependência dos demais microsserviços, sua existência depende dos demais, porém sua implementação foi possível sem a necessidade que os demais estiverem funcionando.

Arquitetura Monolítica	
Os serviços são escalonáveis de forma independente.	Não foi abordado no projeto.
A arquitetura de Microsserviço permite que as equipes sejam autônomas.	Não havia equipes.
Permite fácil experimentação e adoção de novas tecnologias.	Não foi abordado no projeto, porém, cada microsserviço poderia ser implementado em outra linguagem e utilizando outras tecnologias.
Possui melhor isolamento de falhas.	Não foi abordado no projeto.
Desvantagens	Observações
Manter consistência de dados	Correto, manter ACID em uma arquitetura de microsserviços requer estudo e conhecimento do melhor padrão como Saga, CQRS, Event Sourcing. Neste projeto não foi abordado nenhum desses padrões, apenas implementado algo parecido com CQRS onde um método é disparado para marcar um inativo e o porquê. Caso a aplicação tenha continuidade, alguns dos padrões teriam que ser implementados, visando uma melhor organização e mantendo um padrão dentro da aplicação.
Tratamento de erros	Correto, o tratamento tem de ser bem descritivo, de modo que o desenvolvedor possa identificar exatamente onde ocorreu o erro.
Latência	Não foi possível averiguar de fato a latência. Nenhuma ferramenta foi usada para medir tal demora. No entanto, as requisições feitas mesmo em Local, a arquitetura mostrou uma demora no retorno das informações a mais que quando requisitada por uma rota da aplicação monolítica.
Curva de aprendizado	A carga de conhecimento para projetar um sistema com arquitetura de microsserviços é de fato grande. Pensar na comunicação, tratamento de erros, hospedagem, conexões com bancos de dados, balanceadores, escalonar é um processo árduo, pois, cada padrão do microsserviço existe para solucionar algum problema já vivenciado por outros. Distinguir e compreender quais padrões ou como implementá-las se torna uma tarefa demorada e comprometedora.

Quadro 5. Vantagens e desvantagens das arquiteturas monolíticas e de microsserviços.

Em resumo ao Quadro 5, uma comparação direta entre as arquiteturas, a maior dificuldade foi a curva de aprendizado, quando feita a transição entre estas arquiteturas. Pois, de fato, a arquitetura de microsserviços necessita de mais implementações para seu funcionamento correto, como implementação de uma comunicação mais rápida entre os

demais microsserviços, a persistência de dados em banco de dados por cada aplicação e tratamento de erros, que consigam ser claros. Por mais que ambas implementações fazem o uso do mesmo projeto de DDD, a implementação das duas arquiteturas teve uma diferença. Para o desenvolvimento da aplicação monolítica foram gastos 17 horas enquanto a de microsserviços foram gastos 65 horas aproximadamente, estes valores foram tirados de um *plugin* chamado *Waka Time*, instalados na *IDE VSCode*. Claro que estas métricas não caracterizam um padrão geral, representando apenas gasto de tempo pessoal para o desenvolvimento deste projeto.

No entanto, voltando ao tempo gasto, fica claro que a arquitetura de microsserviços demanda mais tempo e conhecimento. Porém, estes valores podem mudar conforme os recursos empregados. Exemplo, tamanho de equipe, conhecimento sobre as tecnologias, padrões que serão usados e tempo disponível, estes fatores influenciam se o sistema será entregue em tempo estabelecido.

O repositório destes códigos podem ser acessados pelos seguintes links:

Primeira ideia:

- Microsserviços: https://github.com/christiancesar/dreams-project

Segunda ideia:

- Monolítico: https://github.com/christiancesar/dreams-project-monolith
- Microsserviços: https://github.com/christiancesar/dreams-project-microservices e https://github.com/christiancesar/dreams-proto-sharing

5 CONCLUSÃO

A Arquitetura de Microsserviços tem uma curva de aprendizagem maior que a Monolítica, enquanto uma aplicação de Microsserviços nos dá mais flexibilidade, manutenibilidade e independência, de outro lado é dificultoso em fazer esses Microsserviços se comunicarem, mantendo-o funcional e provendo uma boa experiência ao usuário.

O desenvolvimento de uma aplicação requer estudo e compreensão sobre o tema a ser desenvolvido, isto é, o desenvolver com os especialistas da área devem estar alinhados em sua Linguagem Ubíqua, consequentemente definir bons Contextos Delimitados e representando isto com o Mapa de Contextos, estas etapas quando bem estruturadas e definidas, geram um sistema preciso e compreensivo pela equipe.

Por mais que a Arquitetura de Microsserviços seja interessante, a decisão de implementar esta Arquitetura deve ser bem estudada, pois, não ter um domínio bem definido, experiência e tempo, possibilita o fracasso do projeto ou que o mesmo se torne mais confuso que um grande Monolítico. Essas conclusões vão de encontro com Fowler (2015), onde um Monolítico ser a melhor opção para se iniciar um primeiro projeto e após, onde o domínio é conhecido, fazer uma transição gradualmente dos módulos dos Monolíticos para

um Microsserviço e Newman (2015), onde afirma que a Arquitetura Monolítica é válida e por mais que as vantagens que a Arquitetura de Microsserviços traz, nenhuma delas é de graça em relação a conhecimento e recurso.

Portanto, este trabalho conclui que fazendo o uso do *Domain Driven Design*, Princípios SOLID, Arquitetura Hexagonal e Padrões de Projeto, independente do paradigma de linguagem de programação ou Arquitetura, facilita no desenvolvimento, pois, traz clareza sobre o projeto que irá ser desenvolvido e, desse modo, se facilita a transição de Arquitetura, pois, delimitando os domínios e modularizando-os, torna o projeto mais coerente. Porém, a escolha da arquitetura de microsserviços há de ser bem analisada, pois, inclui mais conhecimento em relação à implementação.

6 REFERÊNCIAS

AVRAM, A.; MARINESCU, F. Domain-Driven Design Quickly: C4Media, 2009. ISBN: 978-1-4116-0925-9

ERDER, M.; PUREUR, P.; WOODS, E. Continuous Architecture in Practice: Software Architecture in the Age of Agility and DevOps. Pearson, 2021. ISBN: 9780136523796

EVANS, E. Domain-Driven Design: Tackling Complexity in the Heart of Software: Addison-Wesley, 2003. ISBN 0-321-12521-5

FOWLER, M. Design - Who needs an architect?, IEEE, v. 20, n. 5, p.11–13, 2003. ISSN 0740-7459.

FOWLER, M. Patterns of Enterprise Application Architecture: Pearson, 2003. ISBN 0-321-12742-0

FOWLER, M. Monolith First, 2015. Disponível em: https://martinfowler.com/bliki/MonolithFirst.html. Acesso em: 10 Janeiro de 2022.

FOWLER, M. Microservice, 2015. Disponível em: https://martinfowler.com/articles/microservices.htmll. Acesso em: 05 Outubro de 2021.

GERHARDT, T.; SILVEIRA, D. Métodos de pesquisa: Universidade Federal do Rio Grande do Sul, 2009. ISBN 978-85-386-0071-8

MARTIN, R. Clean Architecture: A Craftsman's Guide to Software Structure and Design: Pearson, 2018. ISBN 978-0-13-449416-6

MARTIN, R. Agile Software Development, Principles, Patterns, and Practices, 2018: Pearson, 2014. ISBN 978-1-292-02594-0

MILLETT, S.; TUNE, N. Patterns, Principles, and Practices of Domain-Driven Design: Wrox, 2015. ISBN: 978-1-118-71470-6

NEWMAN, Sam. Building Microservices, 2015. O'Reilly Media, Inc. ISBN: 9781492047841.

NEWMAN, Sam. Monolith to Microservices, 2019. O'Reilly Media, Inc. ISBN: 9781491950357.

RICHARDSON, Chris. Microservices Patterns, 2018. Manning Publications. ISBN: 9781617294549.

RICHARDSON, C; SMITH, F. Microservices: From Design to Deployment, 2016. NGINX. Disponível em: https://www.nginx.com/resources/library/designing-deploying-microservices/. Acesso em: 08 Setembro 2022.

SOMMERVILLE, I.Software engineering. Boston: Pearson, 2011. ISBN 978-0137035151.

VAUGHN, V. Implementing Domain-Driven Design: Pearson, 2013. ISBN 978-0-321-83457-7.